Practical NO.06(C)

Name - Sakshi Vijaykumar Muthiyan

Roll No. -

Title - Build a Tic-Tac-Toe game using reinforcement learning in Python by using following tasks a. Setting up the environment b. Defining the Tic-Tac-Toe game c. Building the reinforcement learning model d. Training the model e. Testing the model

In [1]:
```
import numpy as np
import random
import pandas as pd
# setting up board layout: 0 = Empty, 1 = Agent (X), -1 = Opponent (O)
class TicTacToeGame:
    def __init__(self):
        self.board = np.zeros(9, dtype=int)
    def reset(self):
        self.board = np.zeros(9, dtype=int)
        return self.get_state_str()
    def get_state_str(self):
        return "".join(map(str, self.board))
    def get_moves(self):
        return np.where(self.board == 0)[0]
    def check_game_status(self):
        # 8 winning lines on a 3x3 grid matrix
        win_lines = [[0,1,2], [3,4,5], [6,7,8], [0,3,6], [1,4,7], [2,5,8], [0,4,
        for line in win_lines:
            line_sum = sum(self.board[idx] for idx in line)
            if line_sum == 3: return 1   # Agent wins
            if line_sum == -3: return -1 # Opponent wins
        if 0 not in self.board: return 0 # Tie game
        return None # running
    def step(self, action, player):
        self.board[action] = player
        status = self.check_game_status()
        # rewards configuration mapping
        if status == 1: reward = 1.0
        elif status == -1: reward = -1.0
        else: reward = 0.0
        return self.get_state_str(), reward, status is not None
print("=== Task a & b: Environment and Game Grid Setup Done ===")
```

```
=== Task a & b: Environment and Game Grid Setup Done ===
```

In [2]:
```
# table dictionary to store calculated board state weights
state_value_table = {}
# training hyperparameters matching manual framework
alpha = 0.2     # learning rate
epsilon = 0.2   # exploration factor
gamma = 0.9     # discount factor
def get_value(state_str):
    # if state string is new, initialize with baseline 0.0
    if state_str not in state_value_table:
        state_value_table[state_str] = 0.0
    return state_value_table[state_str]
print("=== Task c: Q-Value Model Table Initialized ===")
print("Initial memory size count:", len(state_value_table))
```

```
=== Task c: Q-Value Model Table Initialized ===
Initial memory size count: 0
```

In [3]:
```
env = TicTacToeGame()
print("=== Task d: Commencing Model Training Iterations ===")
# loop to run 2000 episodes to update value matrix functions
for episode in range(1, 2001):
    state_str = env.reset()
    done = False
    episode_history = [state_str]
    while not done:
        moves = env.get_moves()
        # epsilon-greedy choice matrix selection
        if random.uniform(0, 1) < epsilon:
            action = random.choice(moves)
        else:
            best_val = -float('inf')
            action = moves[0]
            for m in moves:
                env.board[m] = 1
                next_s = env.get_state_str()
                env.board[m] = 0 # undo
                val = get_value(next_s)
                if val > best_val:
                    best_val = val
                    action = m
        state_str, reward, done = env.step(action, 1)
        episode_history.append(state_str)
        if done: break
        # simulating random opponent move (Player O)
        moves = env.get_moves()
        opp_action = random.choice(moves)
        state_str, reward, done = env.step(opp_action, -1)
        episode_history.append(state_str)
    # updating value weights backwards using temporal difference equation
    for s in reversed(episode_history):
        get_value(s)
        state_value_table[s] += alpha * (reward + gamma * get_value(s) - state_v
        reward = state_value_table[s]
    # printing actual verification values
    if episode % 500 == 0:
        print(f"Episode {episode}/2000: Model optimization steps recorded. Learn
```

```
=== Task d: Commencing Model Training Iterations ===
Episode 500/2000: Model optimization steps recorded. Learned combinations count:
1594
Episode 1000/2000: Model optimization steps recorded. Learned combinations count:
2046
Episode 1500/2000: Model optimization steps recorded. Learned combinations count:
2328
Episode 2000/2000: Model optimization steps recorded. Learned combinations count:
2521
```

In [4]:
```
print("=== Learned State Values Table ===")
print("Displaying numeric matrix keys to verify true calculations\n")
pd.set_option('display.float_format', lambda x: '%.4f' % x)
# picking sample states from table dictionary to show actual work logs
sample_keys = list(state_value_table.keys())[:10]
sample_vals = [state_value_table[k] for k in sample_keys]
matrix_df = pd.DataFrame({
```

```
    'Board State Grid String': sample_keys,
    'Calculated Value Score (V)': sample_vals
})
print(matrix_df)
```

```
=== Learned State Values Table ===
Displaying numeric matrix keys to verify true calculations

  Board State Grid String  Calculated Value Score (V)
0              1000-10001                      0.0257
1              0100-10001                      0.0018
2              0010-10001                      0.0000
3              0001-10001                      0.0000
4              0000-11001                      0.0000
5              0000-10101                      0.0000
6              0000-10011                      0.0000
7             1100-1-1001                     -0.1908
8             1010-1-1001                      0.0000
9             1001-1-1001                      0.0803
```

In [5]:
```
print("=== Task e: Running Deployment Policy Verification Check ===")
state_str = env.reset()
done = False
turn_count = 1
# trace the game step-by-step using optimal choices from dictionary
while not done:
    moves = env.get_moves()
    if len(moves) == 0: break
    best_val = -float('inf')
    action = moves[0]
    # agent moves based on max value position
    for m in moves:
        env.board[m] = 1
        next_s = env.get_state_str()
        env.board[m] = 0
        val = get_value(next_s)
        if val > best_val:
            best_val = val
            action = m
    state_str, reward, done = env.step(action, 1)
    # printing actual board layout matrix array changes live
    print(f"Turn {turn_count} -> Agent choices cell index {action}. Current Grid
    if done:
        final_msg = "Agent (Player X) Won!" if reward == 1.0 else "Game Ended in
        print(f"Terminal Status: {final_msg}")
        break
    moves = env.get_moves()
    if len(moves) > 0:
        opp_action = random.choice(moves)
        state_str, reward, done = env.step(opp_action, -1)
        print(f"Turn {turn_count+1} -> Opponent choices cell index {opp_action}.
        if done and reward == -1.0:
            print("Terminal Status: Opponent (Player O) Won!")
            break
    turn_count += 2
print("\nEvaluation metrics trace check complete.")
```

```
=== Task e: Running Deployment Policy Verification Check ===
Turn 1 -> Agent choices cell index 0. Current Grid Array View: [1 0 0 0 0 0 0 0
0]
Turn 2 -> Opponent choices cell index 1. Current Grid Array View: [ 1 -1  0  0  0
0  0  0  0]
Turn 3 -> Agent choices cell index 3. Current Grid Array View: [ 1 -1  0  1  0  0
0  0  0]
Turn 4 -> Opponent choices cell index 8. Current Grid Array View: [ 1 -1  0  1  0
0  0  0 -1]
Turn 5 -> Agent choices cell index 6. Current Grid Array View: [ 1 -1  0  1  0  0
1  0 -1]
Terminal Status: Agent (Player X) Won!

Evaluation metrics trace check complete.
```